

MÓDULO 3 · FASE MILC: PRAXIS · 3 H DE CLASE

3

Construir y programar en MakeCode — del potro emberá probado en el río al prototipo probado

Robótica y sistemas embebidos

LO QUE TE LLEVAS HOY

Tu **prototipo funcional con micro:bit**, construido y probado: el **programa en MakeCode** (traducido de tu diagrama de flujo del módulo 2 y **comentado**), con sus **umbrales medidos de verdad**, probado en **5 escenarios** (resultado esperado / real) con al menos un **ajuste** documentado, y un **video corto** de tu aparato funcionando. Si no tienes micro:bit físico, vale el simulador de MakeCode más el video de la pantalla.

ESTA GUÍA ES DE

Nombre completo

Grupo

Fecha

Autor: Dr. Álvaro Cárdenas Orozco

Metodología MILC · apertura ancestral · pensamiento computacional · triángulo Dussel–estoicismo–Floridi

Apertura: Construir y programar en MakeCode — del potro emberá probado en el río al prototipo probado

Saber ancestral

En los ríos del Chocó —el Pacífico que también es Valle— los hombres emberá son maestros construyendo canoas. El **potro** nace de un *solo tronco* (cedro, chanchazo), labrado con paciencia hasta ahuecarlo (Todacolombia; ONIC). Pero una canoa no se juzga por lo bonita que quedó en la orilla: **se juzga en el agua. El río es el juez.** Si el maestro dejó una pared más gruesa de un lado, la canoa se ladea; si la ahuecó de más, se raja; si la proa quedó torcida, no corta la corriente. **Ninguna de esas fallas se ve en tierra** —todas aparecen en la primera prueba en el río. Por eso el emberá labra sabiendo que su obra será probada por la **realidad**, no por su buena intención. Construir un aparato es igual: un programa que “corre” en la pantalla todavía no es un aparato que *sirve*. La prueba de verdad no es que el código se vea bonito: es que **funcione cuando lo enfrentas al mundo**, escenario por escenario. El potro se prueba en el río; tu prototipo se prueba en la corriente de todo lo que puede salir mal.

Saber contemporáneo

Hoy fundes el oro: conviertes tu **plano del módulo 2** en un aparato que funciona. Y la buena noticia es que **no vas a inventar nada: vas a traducir**. Cada figura de tu diagrama de flujo tiene su bloque en **MakeCode** (makecode.microbit.org): el *óvalo* se vuelve el bloque «**por siempre**» (el aparato vigila sin parar); cada *rectángulo* (una acción) se vuelve un bloque que *lee* un sensor o *muestra* algo; cada *rombo* (una pregunta) se vuelve un bloque «**si...si no**» con tu umbral. Si el plano estaba sin huecos, la traducción sale casi sola. Faltan dos oficios. **Calibrar de verdad:** en el módulo 2 dejaste umbrales con método; hoy los **mides** —tapas el sensor y anotas, lo pones a la luz y anotas— y fijas el corte con esos números reales. **Probar:** como el potro en el río, defines **5 escenarios** con su resultado *esperado*, corres el aparato y anotas el *real*; si alguno falla, **ajustas y vuelves a probar** —y *comentas* tu código para que otro lo entienda. *¿Sin micro:bit físico? El simulador de MakeCode trae los sensores en pantalla: puedes construir y probar igual.*

Pregunta-puente

El emberá no confía en que su canoa sirve hasta verla flotar y cargar en el río; ¿por qué un programa que «corre» en la pantalla todavía no te asegura que tu aparato sirve...y qué prueba de verdad —escenario por escenario— necesitas para confiar en él?

Saber y saber hacer

Al terminar la sesión: (1) podrás **identificar** qué bloque de MakeCode le corresponde a cada figura de tu diagrama de flujo; (2) sabrás **explicar** cómo se calibra un umbral con medidas reales y por qué un programa se prueba en escenarios; (3) podrás **crear** tu prototipo funcional en MakeCode, calibrado y comentado; (4) habrás **evaluado** tu aparato en 5 escenarios (esperado/real), ajustando lo que falle.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Formula preguntas investigables, produce evidencia con método y comunica hallazgos reconociendo sus límites (investigación estudiantil STEM, transversal 6°–11°)
Referentes	micro:bit · MakeCode · Continuidad con la unidad de 8° P2 (guías 8-2-1 a 8-2-10)
Producto final	Tu prototipo funcional con micro:bit , construido y probado: el programa en MakeCode (traducido de tu diagrama de flujo del módulo 2 y comentado), con sus umbrales medidos de verdad , probado en 5 escenarios (resultado esperado / real) con al menos un ajuste documentado, y un video corto de tu aparato funcionando. Si no tienes micro:bit físico, vale el simulador de MakeCode más el video de la pantalla.

→ Escuchaste (módulo 1) y diseñaste el plano (módulo 2). Hoy **construyes**: traduces tu diagrama a bloques, mides tus umbrales de verdad y pruebas el aparato —como el emberá lleva el potro al río.

Ruta MILC: así vamos a aprender

1

Escucha
Observo, escucho
y pregunto.

2

Entender
Sistematizo
y ordeno.

3

Hacer
Construyo y aplico.

4

Cierre
Evalúo y reflexiono.

→ Antes de armar bloques, saca tu plano del módulo 2. Un buen prototipo no empieza en MakeCode: empieza en el diagrama que ya depuraste.

Estación 1 de 3 · Escucha

Escena para escuchar

Actividad 1 · IDENTIFICA — De tu diagrama a los bloques (30 min · individual + grupo).
Momento A (10 min) — El caso del potro. El profe cuenta cómo el emberá prueba su canoa en el río —donde aparecen las fallas que no se ven en tierra— y pregunta: ¿por qué «se ve bien» no es lo mismo que «funciona»?.
Momento B (12 min) — Traducir, no inventar. En grupo, tomen un diagrama de flujo simple y **emparejenlo con sus bloques**: óvalo → «por siempre»; rectángulo «leer» → variable = sensor; rombo → «si...si no»; rectángulo «mostrar» → bloque mostrar.
Momento C (8 min) — Mi plano, listo para traducir. Cada quien saca su diagrama de flujo del módulo 2 y marca, al lado de cada figura, **qué bloque de MakeCode** le tocará. **En el cuaderno:** el emparejamiento figura → bloque + tu diagrama del módulo 2 anotado con sus bloques.

MARCA CUANDO LO HAYAS HECHO

- ☐ Puedo explicar por qué «se ve bien» no es lo mismo que «funciona».
- ☐ Emparejé cada figura del diagrama de flujo con su bloque de MakeCode.

☐ Anoté, sobre mi diagrama del módulo 2, qué bloque le toca a cada figura.

Lo que va al cuaderno (Actividad 1 · IDENTIFICA). Título: «Actividad 1 — De mi diagrama a los bloques». **Formato:** la tabla figura → bloque (óvalo, rectángulo, rombo) + mi diagrama de flujo del módulo 2 con cada figura marcada con su bloque. **Extensión:** 8 líneas. **Sabes que terminaste cuando** tu diagrama del módulo 2 ya dice, figura por figura, qué bloque de MakeCode le corresponde.

→ Ya tienes tu plano. Ahora aprende a traducirlo: qué bloque es cada figura, cómo se mide un umbral de verdad y cómo se prueba en escenarios.

Estación 2 de 3 · Entender

Lo que vas a entender

Actividad 2 · EXPLICA — Traducir, calibrar y probar (55 min · grupo + individual). Ya sabes que programar es traducir. Ahora entiende los **bloques** de MakeCode que necesitas, cómo **medir** un umbral de verdad y cómo se **prueba** un aparato en escenarios —la prueba del río.

1 Pilar 1 — Cada figura, un bloque. MakeCode es visual: se arrastran bloques que encajan. Los que necesitas casi siempre: el bucle «**por siempre**» (tu óvalo: el aparato vigila sin parar); una **variable** para guardar la lectura (*luz = nivel de luz*); la estructura «**si...si no**» (tu rombo: la decisión con tu umbral); y los **actuadores** *mostrar íconos*, *mostrar número*, *reproducir sonido* (tu rectángulo de acción). Para 3 estados se usa «**si...si no si...si no**». **No memorices:** busca cada bloque por su nombre en las gavetas de colores de MakeCode. Si tu diagrama del módulo 2 estaba claro, esto es armar un rompecabezas cuyas piezas ya conoces.

2 Pilar 2 — Calibrar de verdad: del método al número. En el módulo 2 escribiste *cómo* calibrarías; hoy lo **haces**. Con el micro:bit (o el simulador): lee el sensor en la condición «**baja**» (tapado, frío, quieto) y **anota el número**; léelo en la condición «**alta**» (con luz, caliente, agitado) y anota; **pon el umbral entre los dos**, más cerca del que quieras distinguir. “*Tapado me dio 5, con luz 180; pongo el umbral en 40*”. Ese número ya no es un invento: es una medida. **Un umbral bien calibrado es la diferencia** entre un aparato que acierta y uno que se dispara por cualquier cosa.

3 Pilar 3 — Probar es llevar el aparato al río. Un programa que corre no es un aparato que sirve —eso solo lo dice la **prueba**. Se definen **5 escenarios** concretos con su resultado **esperado**, se corre el aparato en cada uno y se anota el **real**. Para un detector de luz: (1) tapado → espero «oscuro»; (2) bajo el bombillo → «con luz»; (3) en penumbra → «intermedio»; (4) tapando de a poco → que cambie en el umbral; (5) moviéndolo cerca de una pantalla → ver si se confunde. **La prueba no busca aplausos: busca fallas.** El escenario que *rompe* tu aparato es el más valioso, porque te dice qué arreglar —como la primera navegada le dice al emberá qué lado rebajar.

4

Pilar 4 — Ajustar, comentar, mostrar. Cuando un escenario falla —y alguno fallará— no es un fracaso: es el oficio. **Ajusta** (casi siempre un umbral o una regla), anota **qué cambiaste y por qué**, y **vuelve a probar**. Ese ciclo probar → ajustar → probar es lo que vuelve confiable un aparato. Dos remates: **comenta tu código** —una nota corta en los bloques clave, para que otro (o tú en un mes) entienda la lógica—, y graba un **video corto** mostrándolo funcionar en un par de escenarios. *El emberá no explica su canoa con palabras: la pone a flotar. Tu video es tu canoa flotando.*

Bloques + calibración + prueba + ajuste

Bloques: «por siempre» (óvalo) · variable = sensor (leer) · «si...si no» con umbral (rombo) · mostrar/sonar (acción).

Calibrar: medir condición baja y alta, poner el umbral entre las dos. Número medido, no inventado.

Probar: 5 escenarios con esperado/real. El que rompe el aparato es el más valioso.

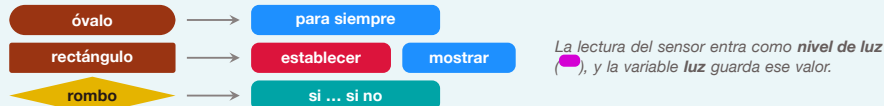
Cerrar: ajustar lo que falle, comentar el código, grabar el video.

Errores comunes y su corrección

Errores al construir y probar (y cómo evitarlos). (1) *Programar sin el diagrama al lado* — te pierdes; **ten tu plano del módulo 2 abierto** y traduce figura por figura. (2) *Umbral a ojo* — «pongo 100 a ver» dispara el aparato por nada; **mídelo**. (3) *Probar solo el caso que sabes que funciona* — eso no es probar; **busca los escenarios que podrían romperlo**. (4) *Rendirse cuando falla* — la falla es la información más útil; **ajusta y reintenta**. (5) *Código sin comentarios* — en un mes ni tú lo entiendes; **deja notas cortas**. (6) *Contaminar la lectura* — una pantalla o el propio brillo del micro:bit cerca del sensor de luz falsea todo; **prueba en condiciones limpias**.



Cada figura de tu diagrama de flujo (módulo 2) tiene su bloque:



El programa del fotómetro, tal como se arma en MakeCode (bloques y etiquetas reales del entorno). Programar en el módulo 3 no es inventar: es traducir el plano del módulo 2 —cada figura del diagrama de flujo tiene su bloque, como muestra la leyenda de abajo. Si el plano estaba bien, la traducción sale sola.

Lo que va al cuaderno (Actividad 2 · EXPLICA). Título: «Actividad 2 — Traducir, calibrar y probar».

Formato: 3 bloques. Bloque A: los bloques de MakeCode que usaré, cada uno frente a su figura del diagrama. Bloque B: mi plan de calibración (mido baja = __, alta = __, umbral = __). Bloque C: mis 5 escenarios de prueba con su resultado esperado. **Extensión:** 1 hoja. **Sabes que terminaste cuando** tienes escritos tus 5 escenarios **antes** de programar (para no probar solo lo que ya sabes que sirve).

→ Ya sabes traducir. Es hora de construir tu aparato, calibrarlo con medidas reales y **llevarlo al río**: los 5 escenarios de prueba.

Estación 3 de 3 · Hacer

Cómo vas a construir tu producto

Actividad 3 · CREA + EVALÚA — Construye, calibra y prueba tu prototipo (85 min · individual o parejas). Hora de fundir el oro. Vas a armar tu aparato en MakeCode traduciendo tu plano, calibrar sus umbrales con medidas reales, probarlo en 5 escenarios y ajustar lo que falle —y grabar tu canoa flotando.

1 Paso 1 — Traduce el plano a bloques (25 min). Abre *makecode.microbit.org* con tu diagrama de flujo del módulo 2 al lado. Arma, figura por figura: el bucle «**por siempre**», la **variable** con la lectura del sensor, la estructura «**si...si no**» con tu umbral, y los **actuadores** (mostrar ícono/número, sonido). Prueba en el **simulador** moviendo el sensor en pantalla mientras armas. **No inventes lógica nueva:** si algo no encaja, el problema suele estar en el plano —vuelve a él, no improvises.

2 Paso 2 — Calibra con medidas reales (10 min). Con el micro:bit (o el simulador), **mide:** la condición baja (tapado/frío/quieto) → anota; la alta (luz/calor/movimiento) → anota; **fija el umbral** entre las dos y actualízalo en tu bloque «si». Escribe la justificación: “*baja = __, alta = __, umbral = __*”. Ahora tu decisión se apoya en números, no en corazonadas.

3 Paso 3 — Comenta tu código (5 min). En los bloques clave, deja una **nota corta** (MakeCode permite comentarios): “*aquí decido si está oscuro*”, “*umbral calibrado el __*”. Un código comentado es un código que otro puede seguir —y que tú entenderás dentro de un mes.

4 Paso 4 — Llévalo al río: 5 escenarios (25 min). Saca tu tabla de 5 escenarios (la escribiste en la Actividad 2). Corre el aparato en cada uno y anota el resultado **real** junto al **esperado**. **¿Coincidieron los 5?** Si uno falla —y busca que falle—, no lo tapes: **ajusta** (casi siempre el umbral o una regla), anota **qué cambiaste**, y **repite el escenario**. Ese ajuste documentado es la joya del módulo: prueba que tu aparato mejoró con evidencia, no con suerte.

5 Paso 5 — Graba tu canoa flotando (10 min). Con un celular, graba un **video corto** (30–60 s) mostrando tu aparato **funcionar** en al menos 2 escenarios: qué percibe, cómo reacciona. No hace falta editarlo; hace falta que se **vea funcionar**. El emberá no describe su canoa: la pone a flotar. Tu video es esa prueba.

6 Paso 6 — Cierra el prototipo (10 min). Reúne todo: el programa (comentado), la tabla de calibración, la tabla de 5 escenarios con su ajuste, y el video. *Esto es un prototipo probado, no un dibujo*. En el módulo 4 vas a **evaluarlo a fondo** —sus fallas, sus límites, su mejora— y a decidir a quién sirve y con qué cuidado. Hoy hiciste lo que el emberá: no dijiste que tu canoa sirve; la **probaste** en el agua.

Antes de dar por bueno tu prototipo

- ☐ Traduje mi diagrama de flujo del módulo 2 a bloques (no inventé lógica nueva).
- ☐ Calibré mis umbrales con medidas reales y escribí la justificación.
- ☐ Comenté los bloques clave del código.
- ☐ Probé los 5 escenarios (esperado/real) y busqué que alguno fallara.
- ☐ Ajusté lo que falló y anoté qué cambié, y volví a probar.
- ☐ Grabé un video corto de mi aparato funcionando en 2 escenarios.

Plantilla de trabajo

Plantilla del prototipo. Calibración: baja = __ · alta = __ · umbral = __.

Bloques: por siempre → variable = sensor → si (umbral) mostrar __ / si no mostrar __.

Pruebas (esperado / real): 1 __ · 2 __ · 3 __ · 4 __ · 5 __.

Ajuste: en el escenario __ falló porque __; cambié __ y volvió a pasar. Video: sí / no.

Lo que va al cuaderno (Actividad 3 · CREA + EVALÚA). Título: «Actividad 3 — Mi prototipo probado».

Formato: captura o esquema del programa comentado + tabla de calibración + tabla de 5 escenarios con el ajuste documentado + enlace o nota del video. **Extensión:** 1-2 hojas. **Sabes que terminaste cuando** tu aparato pasó los 5 escenarios (con al menos un ajuste por el camino) y quedó grabado funcionando.

→ El producto no es «un programa que corre»: es un **prototipo probado** —calibrado, comentado, superando 5 escenarios— con su video. Eso es un aparato que sirve.

Producto de la sesión

Prototipo micro:bit funcional, calibrado, comentado y probado en 5 escenarios (con video).

Criterios de calidad

(1) El programa traduce el diagrama de flujo del módulo 2 (misma lógica, sin huecos). (2) Los umbrales están calibrados con medidas reales y justificados. (3) El código está comentado en sus bloques clave. (4) Los 5 escenarios están documentados (esperado/real) con al menos un ajuste. (5) Hay un video corto que muestra el aparato funcionando.

→ Antes de cerrar, sé honesto: ¿en qué escenario falló y qué ajustaste? Un aparato que nunca falló en la prueba probablemente no se probó bien.

Evaluación liberadora · cinco dimensiones

Sentido de la evaluación

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

→ Cerramos con 3 voces que te ayudan a entender por qué construir y probar —no solo saber— es lo que convierte una idea en un aparato en el que se puede confiar.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

El otro se revela realmente como otro...como el pobre, el oprimido; el que, a la vera del camino, fuera del sistema, muestra su rostro sufriente y sin embargo desafiante: «¡Tengo hambre!, ¡tengo derecho a comer!»

En el módulo 1 escuchaste el reclamo de una necesidad. Hoy **la respondes con las manos**: construir el aparato que sirve es praxis —transformar la realidad, no solo pensarla. Dussel diría que la técnica se justifica cuando *obra* a favor de una vida concreta: la abuela que oír el timbre, el corral que no quedará abierto. **Un prototipo que funciona y sirve es una idea que dejó de ser palabra y se volvió acto.**

¿Construí algo que de verdad responde al reclamo de alguien, o me quedé en la idea?

Estoicismo · Séneca · Cartas a Lucilio, 20 (c. 64 d.C.) · cuidado interior

La filosofía enseña a obrar, no a hablar; quiere que cada uno viva a la manera que prescribe, que estén en armonía nuestras palabras con nuestras acciones.

Séneca desconfía del que solo habla. Tu diagrama del módulo 2 era una promesa; hoy la **cumples con actos**: armar los bloques, medir los umbrales, correr las pruebas. **Un aparato que solo existe en el plano no sirve a nadie; uno construido y probado, sí.** Y cuando un escenario falla, la coherencia estoica no es esconderlo: es ajustar, para que tus actos (el aparato) respalden tus palabras (lo que dijiste que haría).

¿Mi aparato hace de verdad lo que mi plano prometía, o quedó en promesa?

Floridi · lente de la infoesfera

Los pequeños patrones solo pueden ser significativos si se agregan correctamente, se comparan y se procesan a tiempo.

Floridi dice que un dato suelto no significa nada: hay que agregarlo y compararlo. Un aparato que funcionó **una vez** no está probado —pudo ser suerte. Solo al correr **varios escenarios y comparar** esperado con real aparece la verdad sobre tu prototipo: dónde acierta y dónde falla. **La confianza no nace de un funcionamiento afortunado, sino del patrón que forman las cinco pruebas juntas.** Probar es convertir «me funcionó» en «funciona».

¿Confío en mi aparato porque funcionó una vez, o porque lo probé en varios escenarios y comparé?

Nota para el docente. Las tres formulaciones del triángulo son la síntesis del autor de la guía, no citas textuales de los pensadores. Si vas a citarlos en clase, remite a la obra original.

Mi autoevaluación · marca una casilla por fila

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	☐ Explico las ideas con mis palabras.	☐ Reconozco las ideas, falta precisión.	☐ Necesito repasar lo básico.
Pensamiento computacional	☐ Usé los pilares al armar mi producto.	☐ Usé algunos pilares.	☐ Necesito releer la estación 2.
Aplicación y producto	☐ Mi producto cumple los criterios.	☐ Está armado, con ajustes pendientes.	☐ Aún está en construcción.
Reflexión 5 dimensiones	☐ Respondí cada una con honestidad.	☐ Respondí algunas con detalle.	☐ Mis respuestas son muy generales.
Triángulo de pensamiento	☐ Conecté las 3 voces con mi trabajo.	☐ Conecté al menos una.	☐ No las relaciono con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”
Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo:

Fecha: _____ **Curso/grupo:** _____

Mi firma: _____

Para la próxima sesión. Dos cosas. **(1) Deja tu prototipo listo y probado:** programa comentado, umbrales calibrados con sus números, tabla de 5 escenarios con el ajuste documentado, y el video. Si algo aún falla, es normal —anótalo como una falla pendiente; en el módulo 4 la evaluación honesta cuenta más que un aparato perfecto. **(2) Pruébalo con su dueño:** muéstraselo a la persona a quien le serviría (la que validaste en el módulo 1) y **obsérvala usarlo** sin explicarle. Anota qué entendió, qué no, qué le sirvió y qué no —esa es la prueba más dura y más útil. **Trae:** el prototipo, el video, y lo que te dijo su dueño. *En el módulo 4 cierras la línea de Robótica: vas a evaluar tu aparato a fondo —qué mejorarías, sus límites, su ética— y a comunicarlo con honestidad. El emberá ya probó su canoa en el río; ahora falta contar, sin adornos, qué tan bien navega.*